

Unknot recognition: an executable baseline and an audit of the quasi-polynomial specification

Implementation and technical report

17 September 2026

Delivery status: partial implementation. The requested general $n^{O(\log n)}$ implementation has *not* been established. The supplied general recognizer is executable and mathematically exact, but its complete backend has an exponential $2^{O(n)}$ bound. The hierarchy components described below do not constitute a complete implementation of the algorithm announced in the supplied notes.

Abstract

This archive contains a dependency-free Python reference recognizer, three implemented hierarchy-related components, tests, examples, and a complexity audit. The complete recognition path constructs reduced Khovanov homology over $\mathbb{F}_2$, with optional descending-diagram and determinant certificates before it. The independent hierarchy components implement compressed cohomology-dual normal coordinates, essential boundary-pattern testing on a known 3-ball, and bounded lexicographic accounting. We specify their mathematical contracts, explain what is verified, and identify the geometric operations and quantitative invariants still needed for the requested running time. Tests and small-case measurements are reported as such, not as a substitute for a worst-case proof.

1 The target and the source audit

The input is a classical one-component knot diagram with n crossings. The requested result is a two-valued decision procedure, correct on every such input, with worst-case bit complexity

$$\exp(O((\log(n+2))^2)) = n^{O(\log n)}$$

for $n \geq 2$. Returning **unknown** on difficult inputs does not meet that specification. Nor does an exponential algorithm become quasi-polynomial because its tested examples happen to finish quickly.

The supplied February 2021 slides announce this bound on pages 11–14 [1]. Their starting hierarchy algorithm is summarized on page 53. Pages 66–74 identify four accelerations: a quadratic surface-complexity bound, efficient hierarchy encoding, multisurfaces, and generalized Heegaard splittings with Cheeger regions. The last page names the substantial geometric procedures required to implement that strategy. The present work follows the distinction in the slides between the initial hierarchy method and those additional accelerations.

An additional source located during this work is Lackenby’s July 2026 preprint [2]. Section 9 explicitly distinguishes iteration counting from running-time analysis and leaves some operations insufficiently specified for the latter. Section 10 supplies a linear hierarchy-length bound in its

own complexity measure, not the logarithmic bound needed here. This is a statement about that particular source, not a claim that the announced result is false or that a complete implementation is impossible.

The missing implementation is concrete: this archive does not construct the required hierarchical multisurfaces, perform compressed surgery with pattern history, find and use Cheeger regions, or execute the weak reductions in the final flowchart. It therefore supplies neither the entire geometric algorithm nor the combined quantitative proof. Instead of replacing those operations by oracles, the archive separates a complete, slower recognizer from real, individually testable hierarchy components.

2 Executable interface and input validation

The package is **unknotlab**. It requires Python 3.10 or newer and only the standard library. It runs directly from the extracted folder:

```
python -m unknotlab examples/torus_3_5.json --check-d2
python -m unittest discover -s tests -v
python benchmark.py
python hierarchy_demo.py
```

The CLI accepts either a planar-diagram object or a braid-closure object:

```
{"pd": [[1,2,3,4], [4,3,5,6], [6,5,2,1]]}
{"braid": {"strands": 3, "word": [1,-2,1,-2]}}
```

A crossing (a, b, c, d) lists outgoing half-edges in counterclockwise cyclic order. The underpass joins a to c and the overpass joins b to d . Every positive integer arc label must occur exactly twice. Labels are normalized before use. Empty PD means one crossing-free circle; hidden additional crossing-free components are not part of this encoding.

Let α pair the two darts carrying the same arc label, and let σ rotate the four darts of each crossing. The cycles of $\sigma\alpha$ are the faces of the orientable ribbon surface. The parser first checks that joining opposite darts and arc-paired darts gives exactly one knot component. This also makes the projection graph connected. The equation

$$V - E + F = n - 2n + F = 2$$

then verifies that its rotation system is spherical. Incidence data that encode a knot only on a positive-genus supporting surface are rejected.

A braid generator $i > 0$ has the strand at position i over position $i + 1$; negative generators reverse that crossing. The word is read from top to bottom and its endpoints are closed in their original positions. Component count is checked from the permutation. The parser rejects a strand count larger than the word length plus one before allocating a permutation: that many strands cannot be connected by the available crossings.

The ordinary output statuses are **unknot** and **knotted**; both have exit code zero. A selected resource limit or an inconclusive **-fast-only** run gives **unknown**, exit code two. Invalid input gives exit code three. Neither timeout nor resource exhaustion is interpreted as a mathematical certificate. No resource limit is imposed by default, so the complete procedure may consume impractically large resources.

Complexity expressed just in n assumes the conventional arc labels $1, \dots, 2n$. For arbitrary larger labels or irrelevant extra input text, one must also account for the actual input bit length. This parsing cost is not hidden in the crossing count.

3 The complete recognition path

3.1 Polynomial-time sufficient certificates

The first optional test tries every starting dart in both traversal directions. When a traversal meets every crossing first on its overpass, it is descending. Such a diagram is an unknot diagram [7]: after choosing the basepoint, its open arc can be realized with strictly decreasing height, with a closing arc outside the rest of the diagram. The classical straightening of this configuration gives an unknotted circle. The test examines $O(n)$ starts and $O(n)$ crossing encounters for each. Failure of this test says nothing.

The second test joins semiarcs through overpasses to obtain the n Wirtinger arcs. It forms the integral coloring matrix with a row

$$2x_{\text{over}} - x_{\text{under},1} - x_{\text{under},2} = 0$$

for every crossing, adding coefficients when arcs coincide. The absolute value of an $(n-1)$ -square cofactor is the knot determinant $|\Delta_K(-1)|$ [6]. The code uses fraction-free Bareiss elimination, with row pivoting and a check of every exact division. A determinant other than one certifies nontriviality because the unknot has determinant one. *Determinant one is never used as a positive certificate.*

3.2 The explicitly constructed reduced complex

The complete backend uses the Frobenius algebra

$$A = \mathbb{F}_2[x]/(x^2), \quad \deg(1) = 1, \quad \deg(x) = -1,$$

with multiplication and comultiplication

$$\begin{aligned} m(1 \otimes 1) &= 1, & m(1 \otimes x) &= m(x \otimes 1) = x, & m(x \otimes x) &= 0, \\ \Delta(1) &= 1 \otimes x + x \otimes 1, & \Delta(x) &= x \otimes x. \end{aligned}$$

These are the usual cube-of-resolutions maps [3, 4]. For a state $s \in \{0,1\}^n$, the smoothing at (a, b, c, d) is

$$0 : (a, b), (c, d), \quad 1 : (a, d), (b, c).$$

Union-find on semiarcs determines its $k(s)$ circles. The circle containing arc 1 is marked; it is required to carry label x . The other circles carry either 1 or x . Thus the reduced state space has dimension $2^{k(s)-1}$. For the crossing-free circle this dimension is one.

The code changes one state bit from zero to one at each cube edge and computes which two circles merge or which circle splits by their incident semiarcs. It applies m or Δ , copying all unchanged labels. Requiring x on the marked circle gives a subcomplex: a split of a marked x has both new labels x , and a nonzero merge involving a marked x is again labelled x .

Cube signs vanish over $\mathbb{F}_2$. The Frobenius identities make the two composites around every square agree; their sum is zero in characteristic two. Hence $d^2 = 0$. The optional runtime check composes the implemented differential on every generator and verifies this cancellation directly.

For a labelling with r copies of x , the package uses

$$h = |s|, \quad q = h + k(s) - 2r.$$

The differential increases h by one and preserves q . These are deliberately *unnormalized* gradings. Orientation-dependent global shifts are unnecessary for total rank; the program does not advertise a normalized Khovanov-polynomial interface.

3.3 Exact linear algebra and the decision theorem

For each (h, q) , a differential column is a nonnegative Python integer whose set bits name nonzero rows. Gaussian elimination stores one column for each highest set-bit pivot; elimination is exact XOR. Let $c_{h,q}$ be the dimension of a chain group and $r_{h,q}$ the rank of its outgoing differential. Then

$$\beta_{h,q} = c_{h,q} - r_{h-1,q} - r_{h,q}, \quad \beta = \sum_{h,q} \beta_{h,q} = C - 2 \sum_{h,q} r_{h,q}.$$

The implementation checks nonnegativity and agreement of these two total-rank calculations. Its decision is $\beta = 1$ for the unknot, $\beta > 1$ for a nontrivial knot.

Theorem 3.1 (Mathematical correctness of the complete backend). *For a valid classical one-component diagram D ,*

$$D \text{ represents the unknot} \iff \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(D; \mathbb{F}_2) = 1.$$

Consequently the specified full-complex computation decides unknotness.

Proof. Kronheimer–Mrowka prove that reduced Khovanov homology of a knot has rank one over $\mathbb{Q}$ precisely for the unknot [5]. The reduced integral complex is free in each chain degree. The universal coefficient theorem implies

$$\dim_{\mathbb{Q}} \widetilde{\text{Kh}}(D; \mathbb{Q}) \leq \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(D; \mathbb{F}_2).$$

The rational reduced homology is nonzero: its graded Euler characteristic is a shifted reduced Jones polynomial, which is nonzero since its normalized value at one is one. Therefore dimension one over $\mathbb{F}_2$ forces rank one over $\mathbb{Q}$, and the cited detector gives the unknot. Conversely the unknot’s reduced complex has one generator, so its reduced homology over $\mathbb{F}_2$ has dimension one. The implemented complex is this standard complex, up to grading shifts and the diagram convention, neither of which changes its total dimension. $\square$

This correctness argument uses established topology. It is not a replacement for the Kronheimer–Mrowka proof, nor a mechanically checked verification of all Python instructions. The code does not infer triviality from the Jones polynomial, determinant, or a table of named knots.

3.4 The actual upper bound is exponential

Proposition 3.2. *The full-complex reference backend has bit complexity $2^{O(n)}$ and uses at most $2^{O(n)}$ bits of storage. It explicitly enumerates 2^n resolutions.*

Proof. Smoothing a crossing can increase the number of components by at most one. Starting with one knot component gives $k(s) \leq n + 1$. Thus

$$C = \sum_{s \in \{0,1\}^n} 2^{k(s)-1} \leq 2^n 2^n = 4^n.$$

Each generator has at most $2n$ differential terms. Resolutions, incidence maps and label maps require polynomial work in n per state or term. A conservative bound for elimination is $O(C^3 \text{poly}(n))$ bit operations: there are at most C columns, each uses at most C pivot eliminations, and an XOR handles at most C row bits. Stored pivots use at most $O(C^2 \text{poly}(n))$ bits. These bounds are exponential in n . The optional d^2 check performs at most polynomially many differential compositions per generator and does not change the exponential upper bound. $\square$

The explicit cube enumeration rules out advertising the full-complex routine as quasi-polynomial. Polynomial-time filters do not constitute a proof that this fallback will be avoided on all large inputs. Resource caps make it a sound partial procedure, not a faster total recognizer.

4 Compressed cohomology-dual normal surfaces

4.1 A concrete finite triangulation format

For tetrahedron t and its face opposite vertex f , a gluing is either a boundary marker or a pair (u, p) , where p permutes $\{0, 1, 2, 3\}$ and carries face f to face $p(f)$ in tetrahedron u . Reciprocal pairings are mandatory. Different faces of one tetrahedron may be identified; a face cannot be paired to itself.

The validator tracks global vertices and signed global edges. Identifying an edge to its reverse is rejected. It checks a consistent orientation on the tetrahedra. Vertex links are constructed as glued triangles; the valid edge-pairing conditions give connected surface links. A closed link must have Euler characteristic two and a link with boundary must have characteristic one. Orientability then identifies them as spheres or disks. Positive-genus ideal links are not accepted: this is a finite compact-manifold format, not an ideal-triangulation interface.

4.2 Computing cohomology directions

An integral edge cochain z is closed when, on every triangular face,

$$z_{ab} + z_{bc} - z_{ac} = 0.$$

Global edge orientations are included in these equations. A maximal spanning forest in the vertex-edge graph fixes the gauge by making z vanish on its tree edges. Every rational cohomology class has a unique representative in this gauge: an exact cochain can successively cancel the values on tree edges, and a remaining exact tree-zero cochain vanishes everywhere.

Exact rational row reduction computes the kernel of the face equations in non-tree variables. Clearing denominators and dividing a coordinate gcd gives integral vectors whose classes form a basis over $\mathbb{Q}$. The package does *not* assert that these vectors form a saturated $\mathbb{Z}$ -basis. In a matrix of size $O(t)$ with bounded integer entries, determinant bounds control the bit length of the rational row-reduction data polynomially in t ; this step does not require iterating over the values of its output coordinates.

4.3 The normal vector without disk expansion

In a tetrahedron set $h_0 = 0$ and $h_i = z_{0i}$ for $i = 1, 2, 3$. Closedness gives $z_{ij} = h_j - h_i$. Regard the affine height function modulo $\mathbb{Z}$. Across a paired face the two local height functions differ by an integer constant, so their half-integral level sets agree.

Sort the four heights, breaking ties by vertex name. For $j = 1, 2, 3$, let

$$d_j = h_{v_{j+1}} - h_{v_j} \geq 0.$$

Exactly d_j half-integral levels separate the lowest j vertices from the rest. For $j = 1$ they are triangles around v_1 ; for $j = 3$ they are triangles around v_4 ; and for $j = 2$ they are quadrilaterals of a single type. The result is a seven-entry vector in the order

$$(t_0, t_1, t_2, t_3, q_{01|23}, q_{02|13}, q_{03|12}).$$

Only a constant number of integer comparisons and subtractions occurs per tetrahedron. There is no loop over the d_j individual disks. The three normal arc counts on each paired face are checked explicitly; at most one quadrilateral type per tetrahedron is allowed.

Proposition 4.1. *For a closed integral cochain z on a valid input triangulation, this construction gives a properly embedded, cooriented normal surface dual to z , possibly disconnected or empty. Its coordinates are computed in time polynomial in the triangulation size and the bit length of z .*

Proof. The map to $\mathbb{R}/\mathbb{Z}$ just described is well-defined because the height changes across gluings are integral. The half-integral fiber avoids vertices. On each tetrahedron it consists of parallel planar disks, of exactly the types and multiplicities counted above. The normal matching check verifies their gluing across faces. Increasing height supplies a consistent coorientation; the integral intersection number with an oriented edge is its cochain value. This is the cohomology-duality characterization of the fiber. The sorting and integer operations have the stated bit cost. $\square$

Euler characteristic is also obtained without expansion. If F is the total number of normal disks, E the sum of normal arc counts over global faces (counting a paired face once), and $V = \sum_e |z(e)|$, then

$$\chi(S) = V - E + F.$$

These are the vertex, edge and face counts of the induced surface cell structure, including boundary cells.

In the one-tetrahedron solid-torus example, the computed cocycle is $(3, -1, -2)$, with normal vector $(1, 1, 0, 0, 0, 0, 1)$ and Euler characteristic one. Scaling the cocycle by 10^{1000} produces $3 \cdot 10^{1000}$ normal disks but still only seven integer coordinates. This is a compression demonstration, not a genus bound or a full implementation of the Agol–Hass–Thurston methods mentioned in the notes.

4.4 The contract deliberately stops here

A nonzero cohomology direction gives a nonzero relative homology class, but this implementation does not extract a connected nonseparating component. Different basis directions are not assembled into a controlled simultaneous multisurface. The code does not cut the manifold, transport its boundary pattern, retain its embedding in S^3 , or establish the quadratic complexity bound of the notes. A short binary normal vector by itself proves none of those properties.

5 Essential boundary patterns on a known ball

The source definition asks whether every properly embedded disk meeting the pattern at most three times has an allowed boundary disk: empty pattern, one arc, or one tripod [1, pp. 24–25]. The implementation handles this terminal predicate when the ambient manifold is already known to be a 3-ball. It does not decide whether an arbitrary manifold is a ball.

A trivalent pattern is represented by three cyclically ordered darts per vertex and a perfect matching of darts into edges. Loops and parallel primal edges are permitted. The permutation of face walks determines the spherical embedding, which is checked component by component. Isolated circle components are counted separately.

An empty pattern and one simple closed curve are essential. A disconnected nonempty pattern is inessential, regardless of its nesting on the sphere. For a connected trivalent graph, construct the embedded dual graph. The following exhaustive short-cycle test is implemented:

1. A dual loop supplies a violating one-edge cycle.
2. Two distinct dual edges with the same endpoints supply a violating two-edge cycle.
3. Otherwise enumerate dual triangles. A triangle that is not the boundary of a dual face on either side supplies a violating three-edge cycle.

The output records the dual vertices and crossed primal edges. The two-face triangulation with dual graph K_3 is handled correctly; its triangles are facial, so it is not incorrectly rejected by an unqualified 4-connectivity criterion. The alternative characterization is a simple 4-connected dual, or K_3 or K_4 [2, Proposition 2.6].

Here is a direct justification of the implemented cycle criterion. For a connected pattern, a disjoint curve bounds an empty-pattern disk on one side. After dual loops and parallel edges are excluded, a curve crossing the pattern twice either gives an already excluded pair of distinct dual edges or crosses the same primal edge twice, isolating an arc. A length-three dual walk in a simple graph must be a triangle. Its curve isolates a tripod exactly when the triangle is facial on one side. Thus a nonfacial triangle is the remaining obstruction. Conversely each recorded short cycle can be realized by a simple curve in the indicated spherical faces and pushed into the ball to span a disk. The test is polynomial; a conservative bound for triangle enumeration is cubic in the explicitly represented graph size.

The independent test implementation instead checks graph simplicity and vertex connectivity by enumerating vertex separators of size at most three. Agreement between these differently structured predicates is tested on all four-vertex dart matchings and on larger sampled spherical patterns.

A short dual cycle is not yet the compression disk in an earlier manifold. Recovering that disk through all cuts, with its pattern history and compressed representation, is a separate unimplemented operation. Also, $b_1(M) = 0$ alone does not establish that M is a ball: for example $S^2 \times [0, 1]$ has that Betti number but is not a ball. A full hierarchy implementation must maintain the topological hypotheses behind its terminal branch, rather than silently promote a homology test into ball recognition.

6 What a genuine quasi-polynomial proof must control

6.1 The exact arithmetic part

The slides' digit-counting argument is schematic. For an actual counter with retained digits $0 \leq a_i \leq g$, pad the hierarchy to a fixed length L and use

$$B = g + 1, \quad \Phi(a) = \sum_{i=1}^L a_i B^{L-i}, \quad 0 \leq \Phi(a) \leq B^L - 1.$$

The distinction between g and $g + 1$ matters: base g cannot accommodate a digit equal to g with the required strictness.

Lemma 6.1 (Suffix rebuilding). *Suppose the prefix through index $j - 1$ is unchanged, the digit at j decreases by at least one, and every later digit remains between zero and g . Then the potential decreases by at least one, even if the entire suffix is rebuilt.*

Proof. The loss at j is at least B^{L-j} . The greatest possible gain in the suffix is

$$g \sum_{r=0}^{L-j-1} B^r = B^{L-j} - 1.$$

Subtracting gives a net decrease of at least one. For $g = 0$ no such strict update can occur, so the assertion is vacuous. $\square$

HierarchyBudget evaluates this potential and checks a proposed update. Comparisons are between corresponding completed states, not every individual extension of a partial hierarchy. This distinction avoids falsely demanding that appending a new positive digit lower the potential.

6.2 The conditional running-time statement

Proposition 6.2 (A sufficient quantitative contract). *Assume an implemented geometric algorithm has at most $R(n)$ phases, at most $L(n)+1$ elementary calls between comparable completed states, and a strict potential drop between successive nonterminating completed states of each phase. Assume its digits are bounded by $g(n)$ and its elementary calls, including manipulation of all encoded data, cost at most $P(n)$ bit operations. Then its work is bounded by*

$$R(n)P(n)(L(n) + 1)(g(n) + 1)^{L(n)}.$$

In particular, polynomial R, P, g and logarithmic L imply the requested quasi-polynomial bound.

Proof. There are at most $(g + 1)^L$ completed states in a strict nonnegative descent phase. Multiply this count by the stipulated work between them and by the number of phases. If $g \leq n^c$ and $L \leq C \log(n + 2)$, the logarithm of the exponential factor is $O((\log(n + 2))^2)$. The remaining polynomial factors do not change this order. $\square$

This proposition is a contract, not a proof that the missing topology satisfies it. In particular, an integer returned by `conditional_operation_bound` does not certify the supplied values of L, g, R, P .

6.3 The unresolved geometric and bit-complexity obligations

Requirement	What an implementation and proof still need to provide
Initial encoded geometry	A finite layered handle structure for the knot exterior, a generalized Heegaard/Morse description, boundary-pattern data, and the ambient information needed by the bounded-complexity surface construction. This archive has no PD-to-layered-exterior pipeline.
Hierarchical multisurfaces	Simultaneous properly embedded oriented surfaces with independent classes, controlled intersections, the required pattern-complexity bounds, and the compressed combinatorial data needed by subsequent cuts. Separate cohomology vectors are not enough.
Compressed modifications	Cutting, component selection, compression and pattern-compression without expanding large normal coordinates, together with exact identification maps back through the hierarchy and correct retention or discarding of later surfaces.
Cheeger regions and reduction	A constructive implementation of the alternatives on pages 101–109 of the notes, including modifications of the generalized Heegaard splitting, Haken-lemma and weak-reduction steps, with a quantitative progress measure for restarts.
Combined bounds	A logarithmic hierarchy-depth bound valid after every rebuild, bounded digit complexity, bounded representation size, and a bound for all phases and subroutine bit costs. None follows merely from naming the operations in a flowchart.

The three implemented hierarchy components address only parts of this table. They are not treated as an oracle-based realization of the whole algorithm.

7 Verification results and reproducibility

The delivered version passed 64 `unittest` test methods, including many subcases. The core tests check diagram rejection, exact determinant arithmetic against the Leibniz formula, binary elimination against dense row elimination, and an independently structured small-diagram homology implementation. The latter uses graph walks instead of union-find, label tuples instead of masks, and dense byte matrices instead of packed columns.

Topological consistency tests include crossing-preserving rotations, marked arc changes, mirror examples, generator cancellation, braid conjugation, the braid relation, and positive and negative Markov stabilization. They also check $d^2 = 0$, both recognition outcomes with determinant one, and the non-verdict semantics of resource limits. These are finite tests, not a formal verification of all inputs.

For boundary patterns, all 10,395 perfect matchings of the 12 darts at four trivalent vertices are generated; nonspherical rotation systems are rejected, and accepted systems are checked against the independent connectivity criterion. Larger randomized systems and explicit spherical bipyramids test separating triangles. Normal-coordinate tests include all 24 relabellings of the solid-torus tetrahedron, face matching, sign reversal, and huge compressed coordinates.

Fifteen example diagrams were run through the default recognizer and also through full homology with the d^2 check enabled. A selection of actual outputs is shown below. Here n is the crossing count of the supplied presentation, not necessarily the crossing number of the knot.

Example	n	Determinant	$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}$	Generators
Crossing-free circle	0	1	1	1
Trefoil	3	3	3	15
Figure-eight	4	5	5	33
Conjugated unknot	6	1	1	369
$T(3, 4)$	8	3	5	801
$T(3, 5)$	10	1	7	4209
Alternate 10_{124} braid	10	1	7	14973

The conjugated unknot is the closure of $\sigma_2^{-2}(\sigma_1\sigma_2)\sigma_2^2$. Undoing the conjugation and two Markov destabilizations proves that it is trivial independently of the calculation. The descending test does not recognize this particular diagram, so it exercises the positive fallback path. The $T(3, 5)$ example exercises the negative fallback path and prevents treating determinant one as a complete test.

Six independent braid/determinant data pairs were obtained through Wolfram `KnotData` [8]. The returned data and the expression that regenerates them are stored in `results/`. The runtime has no Wolfram dependency. In this environment the code itself was tested locally with CPython 3.13; no Lean checking or remote Python execution is claimed.

The selected $T(3, 5)$ presentation used 1024 cube states and 4209 generators; full homology plus d^2 checking took approximately 0.078 seconds in the recorded run. The alternate presentation had the same answer but 14973 generators and approximately 0.299 seconds. These timings are illustrative measurements of very small inputs. They give no evidence that the general algorithm meets the requested asymptotic target.

The files `results/tests.log`, `results/benchmark.json`, and `results/benchmark.log` contain the actual test and measurement output. Running `benchmark.py` regenerates the benchmark files, including environment information. The mathematical specification above and `STATUS.json` are explicit about what this delivery does and does not establish.

8 Conclusion

The archive contains real executable recognition code and useful, independently tested pieces of the hierarchy approach. The general recognizer is exact but exponential, and the geometric pieces remain disconnected from the bounded-depth construction. Accordingly, this delivery is not a solution to the requested $n^{O(\log n)}$ implementation. The remaining work consists of the stated constructions and quantitative proofs, not of renaming the fallback or replacing those constructions by unimplemented calls.

References

- [1] Marc Lackenby, *Unknot recognition in quasi-polynomial time*, February 2021, 109-page talk supplied with the request as `quasipolynomial-talk.pdf`. Page references in this report refer to that specific attachment, including its repeated build-up slides.
- [2] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. <https://arxiv.org/abs/2607.23350>.
- [3] Mikhail Khovanov, *A categorification of the Jones polynomial*, Duke Mathematical Journal **101** (2000), 359–426. <https://arxiv.org/abs/math/9908171>.
- [4] Dror Bar-Natan, *On Khovanov’s categorification of the Jones polynomial*, Algebraic & Geometric Topology **2** (2002), 337–370. <https://arxiv.org/abs/math/0201043>.
- [5] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. <https://arxiv.org/abs/1005.4346>.
- [6] Sudipta Kolay, *Knot Colorings: Coloring and Goeritz matrices*, arXiv:1910.08044, 2019. <https://arxiv.org/abs/1910.08044>.
- [7] Maciej Mroczkowski, *Diagrammatic unknotting of knots and links in the projective space*, arXiv:math/0207302, 2002. <https://arxiv.org/abs/math/0207302>.
- [8] Wolfram Research, *KnotData*, Wolfram Language documentation and live braid/determinant data retrieved for this implementation. <https://reference.wolfram.com/language/ref/KnotData.html>.